AI-Powered Multi-Agent Energy Research System

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Ansh Upadhyay - EN22CS301147
Anuroop Patidar - EN22CS301176
Arnika Jain - EN22CS301196
Arnoday Soni - EN22CS301197
Aryan Singh - EN22CS301214

Under the Guidance of

Prof. Akshay Saxena
Prof. Hemlata Patel
Prof. Ajeet Singh Rajput

MEDICAPS UNIVERSITY

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

AI-Powered Multi-Agent Energy Research System

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Ansh Upadhyay - EN22CS301147
Anuroop Patidar - EN22CS301176
Arnika Jain - EN22CS301196
Arnoday Soni - EN22CS301197
Aryan Singh - EN22CS301214

Under the Guidance of

Prof. Akshay Saxena
Prof. Hemlata Patel
Prof. Ajeet Singh Rajput

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Report Approval

The project work **"AI-Powered Multi-Agent Energy Research System"** is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name: Prof. Akshay Saxena
Prof. Hemlata Patel
Prof. Ajeet Singh Rajput

Designation: Faculty of Engineering, Medicaps University Indore

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

I/We hereby declare that the project entitled **"AI-Powered Multi-Agent Energy Research System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications in Computer Science & Engineering completed under the supervision of **Prof. Akshay Saxena, Prof. Hemlata Patel, Prof. Ajeet Singh Rajput,** Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Ansh Upadhyay - EN22CS301147

Anuroop Patidar - EN22CS301176

Arnika Jain - EN22CS301196

Arnoday Soni - EN22CS301197

Aryan Singh - EN22CS301214

Certificate

We, **Prof. Akshay Saxena, Prof. Hemlata Patel, Prof. Ajeet Singh Rajput** certify that the project entitled "**AI-Powered Multi-Agent Energy Research System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications by **Ansh Upadhyay, Anuroop Patidar, Arnika Jain, Arnoday Soni, Aryan Singh** is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Akshay Saxena	<Name of External Guide (If any)>
Computer Science & Engineering	<Name of the Department>
Medicaps University, Indore	Name of the Organization

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to Honorable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya,** Associate Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

We express my heartfelt gratitude to our Internal Guide, , Prof. Akshay Saxena, Prof. Hemlata Patel, Prof. Ajeet Singh Rajput, without whose continuous help and support, this project would ever have reached to the completion.

Ansh Upadhyay - EN22CS301147
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Anuroop Patidar - EN22CS301176
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Arnika Jain - EN22CS301196
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Arnoday Soni - EN22CS301197
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Aryan Singh - EN22CS301214
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

The rapid growth of digital information has made manual research increasingly time-consuming and inefficient. This project presents the design and implementation of **an AI-powered Energy Research Application** that automates the process of web-based research, analysis, and report generation.

The system leverages **Generative Artificial Intelligence (GenAI)** and **Agentic AI** architecture, where multiple specialized agents collaborate to perform tasks such as web searching, content scraping, data analysis, and report summarization. The application integrates modern technologies including React (Vite) for frontend development, Node.js (Express) as an API gateway, and FastAPI (Python) for backend processing. Advanced AI models such as HuggingFace Transformers (Flan-T5) and Sentence Transformers are used for natural language processing and semantic search.

The system also incorporates **FAISS** (Facebook AI Similarity Search) for efficient vector-based retrieval and uses **JSON-based storage** for maintaining research history. To ensure scalability and real-world applicability, the application is containerized using **Docker** and deployed on **AWS EC2** with a fully automated **CI/CD** pipeline using **GitHub Actions**.

The developed system successfully automates the research workflow, reduces manual effort, and generates structured reports from user queries. The project demonstrates the integration of AI, web technologies, and DevOps practices into a unified, scalable, and production-ready system.

Keywords:

Artificial Intelligence, Generative AI, Multi-Agent Systems, Web Scraping, Natural Language Processing, Semantic Search, FAISS, Docker, CI/CD, AWS EC2

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii - viii
	List of figures	ix
	List of tables	x
	Abbreviations	xi
Chapter 1	Introduction (Whichever is applicable)	1-3
	1.1 Introduction	1
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	2
	1.5 Research Design	2
	1.6 Source of Data	2
	1.7 Chapter Scheme	3
Chapter 2	(Report on Present Investigation)	4-7
	2.1 System Architecture	4
	2.2 Technology Stack	6
	2.3 System Workflow	6
Chapter 3	Implementation	8-10
	3.1 Frontend Implementation	8
	3.2 Backend Implementation	8
	3.3 AI Agent Implementation	9
Chapter 4	DevOps & Deployment	11-14
	4.1 Dockerization	11
	4.2 CI/CD Pipeline (GitHub Actions)	12
	4.3 AWS EC2 Deployment	13
Chapter 5	Database & Storage	15-16
	5.1 FAISS Vector Database	15

	5.2 JSON Storage System	16
	5.3 Data Flow Management	16
Chapter 6	Results and Discussions	17-20
	6.1 System Output	17
	6.2 Performance Analysis	20
	6..3 Challenges Faced	20
Chapter 7	Summary and Conclusions	21
	7.1 Summary	21
	7.2 Conclusion	21
Chapter 8	Future scope	22
	Appendix	23
	Bibliography	23

List of Figures

Figure No.	Title
Fig. 2.1	Overall System Architecture
Fig. 2.3	System Workflow Diagram
Fig. 6.1	User Interface – Query Input Page
Fig. 6.2	User Interface – Query Processing Page
Fig. 6.3	Generated Research Report Output (Part A)
Fig. 6.4	Generated Research Report Output (Part B)
Fig. 6.5	AWS EC2 Instance Dashboard
Fig. 6.6	Docker Containers Running

List of Tables

Table No.	Title
Table 1.1	Comparison of Traditional vs AI-Based Research Systems
Table 2.1	Technology Stack Overview
Table 3.1	API Endpoints Description
Table 5.1	JSON Report Structure
Table 6.1	Performance Analysis Metrics

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
GenAI	Generative Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
UI	User Interface
UX	User Experience
CI/CD	Continuous Integration / Continuous Deployment
EC2	Elastic Compute Cloud
FAISS	Facebook AI Similarity Search
NLP	Natural Language Processing
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation

Chapter 1: Introduction

1.1 Introduction

In the modern digital era, the volume of information available on the internet has grown exponentially. While this abundance of data provides immense opportunities for knowledge discovery, it also introduces challenges in efficiently extracting, analyzing, and summarizing relevant information. Traditional research methods require significant manual effort, time, and domain expertise to gather and process information from multiple sources.

To address these challenges, the **Energy Research App** is developed as an intelligent, AI-powered system that automates the process of web-based research and report generation. The application leverages **Generative Artificial Intelligence (GenAI)**, **Agentic AI (multi-agent systems)**, and modern **web technologies** to simulate the workflow of a human research analyst. Additionally, the system is deployed using **Docker containers**, automated through **CI/CD pipelines using GitHub Actions**, and hosted on **AWS EC2**, demonstrating real-world DevOps practices.

The system accepts a user-defined query related to energy or general research topics and performs the following operations:

- Searches the web for relevant information
- Scrapes content from multiple sources
- Filters and analyzes the collected data
- Generates a structured research report using a Large Language Model (LLM)
- Stores the results for future retrieval and semantic search

1.2 Literature Review

Traditional search engines such as Google provide access to vast amounts of data but require users to manually filter, interpret, and compile information.

More recently, AI-based tools such as conversational agents and research assistants have emerged. These systems utilize Large Language Models (LLMs) to generate human-like responses and summaries. However, most existing solutions exhibit certain limitations:

- Lack of end-to-end automation of the research pipeline
- Limited ability to scrape and analyze real-time web data
- Absence of multi-agent collaboration mechanisms
- Minimal support for structured report generation
- No persistent storage and semantic retrieval of research outputs

Research in Agentic AI systems has introduced the concept of dividing complex tasks into smaller subtasks handled by specialized agents.

1.3 Objectives

The primary objectives of this project are as follows:

- To design and develop an automated system for web-based research
- To implement a **multi-agent AI architecture** for task distribution
- To integrate **Generative AI models** for report generation
- To enable semantic search using vector embeddings
- To develop a user-friendly interface for interaction and visualization
- To deploy the system using **Docker and AWS cloud infrastructure**
- To implement a **CI/CD pipeline** for continuous integration and deployment

1.4 Significance

The significance of this project lies in its ability to combine multiple advanced technologies into a single functional system. The key contributions include:

- **Automation of Research Process:** Reduces manual effort and time required for gathering and analyzing information
- **Integration of AI Technologies:** Demonstrates practical use of Generative AI and Agentic AI
- **Scalable Architecture:** Designed using modular components for easy scalability and maintenance
- **Real-World Deployment:** Implements industry-standard DevOps practices such as containerization and CI/CD

1.5 Research Design

The system follows a modular and layered design approach, ensuring separation of concerns and ease of development.

Design Approach:

- **Frontend Layer:** Handles user interaction and displays results
- **API Layer:** Acts as a communication bridge between frontend and backend
- **Backend Layer:** Processes requests and manages AI operations
- **Agent Layer:** Executes specialized tasks such as research, analysis, and summarization
- **Storage Layer:** Stores reports and embeddings

1.6 Source of Data

The data used in this system is collected dynamically from online sources. The primary sources include:

- **Search Engines:** DuckDuckGo for retrieving relevant links

- **Web Pages:** Scraped using BeautifulSoup and HTTP requests
- **Fallback Sources:** Wikipedia for reliable information when search fails

The collected data is processed and filtered before being passed to the AI models for analysis and summarization.

1.7 Chapter Scheme

The report is organized into multiple chapters, each focusing on a specific aspect of the project:

- **Chapter 1:** Introduction, objectives, and overview of the system
- **Chapter 2:** System architecture, design, and workflow
- **Chapter 3:** Implementation details of frontend, backend, and AI agents
- **Chapter 4:** DevOps practices including Docker, CI/CD, and deployment
- **Chapter 5:** Database and storage mechanisms
- **Chapter 6:** Results and discussion of system performance
- **Chapter 7:** Summary and conclusions
- **Chapter 8:** Future scope and enhancements

Chapter 2: System Design & Investigation

2.1 System Architecture

The Energy Research App is designed using a modular, scalable, and microservices-based architecture that integrates frontend interaction, API handling, and AI-driven processing.

The system is divided into three major layers:

1. Presentation Layer (Frontend)

- Responsible for: Accepting user queries, Displaying research progress, Rendering structured reports
- Communicates with backend via HTTP APIs

2. Application Layer (API Server)

- Acts as a middleware/proxy layer
- Handles: Routing (/api/*), Request forwarding, CORS and security

3. Intelligence Layer (AI Backend)

- Implements Agentic AI architecture
- Consists of:
    - **Research Agent** → performs web search
    - **Scraper Tool** → extracts content
    - **Analysis Agent** → processes data
    - **Summary Agent** → generates final report using LLM

4. Data Layer

- FAISS → vector storage for semantic search
- JSON storage → stores generated reports
- Embeddings → Sentence Transformers

5. Deployment Layer

- Dockerized services
- Hosted on AWS EC2
- CI/CD via GitHub Actions

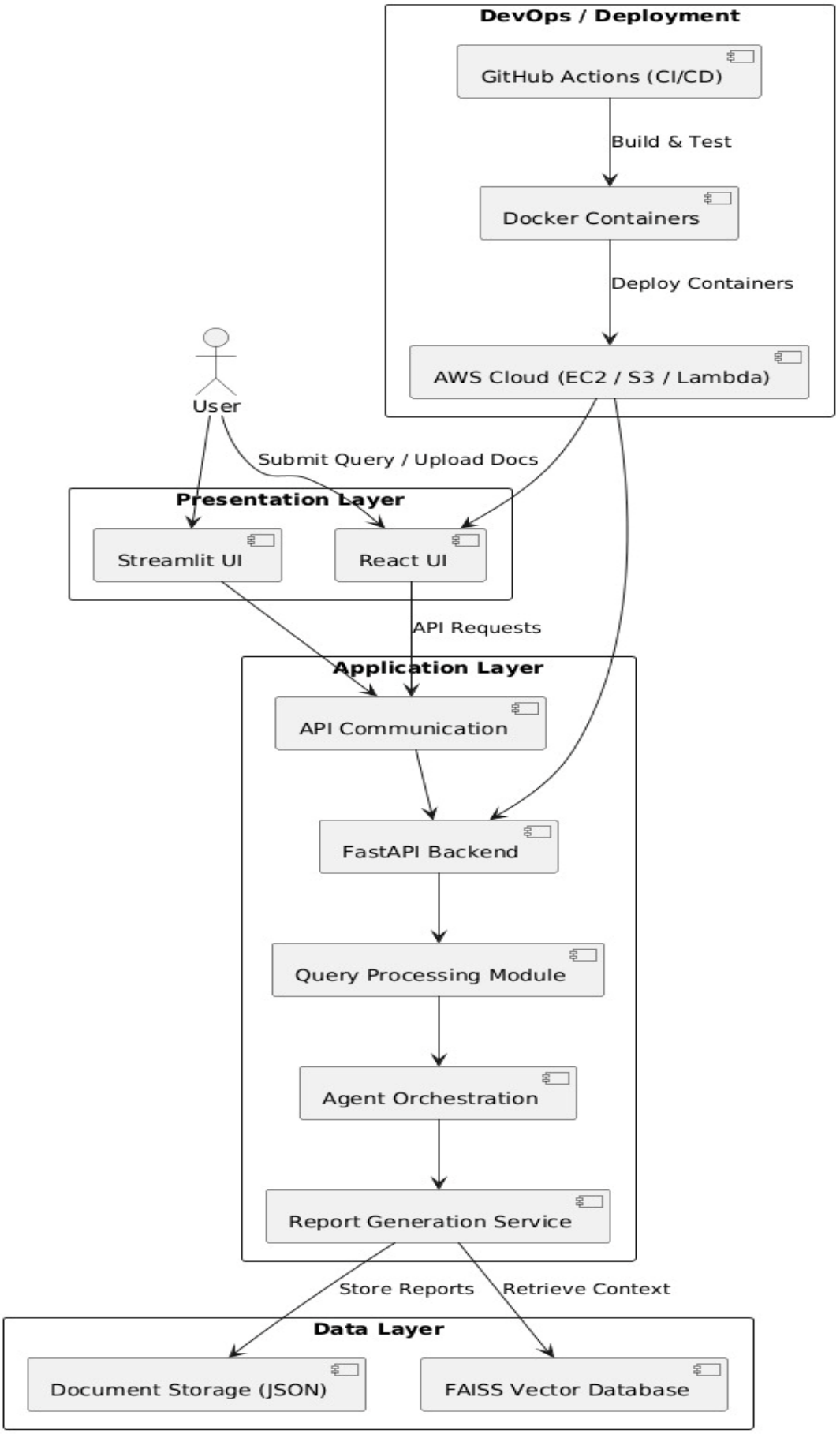

Fig. 2.1: Overall System Architecture

2.2 Technology Stack

The system integrates modern technologies across frontend, backend, AI, and DevOps layers.

Frontend Technologies - React (Vite), Tailwind CSS, TypeScript

Backend Technologies - Node.js (Express) – API Gateway, FastAPI – AI backend

AI & Machine Learning - HuggingFace Transformers (Flan-T5), Sentence Transformers (Embeddings), FAISS (Vector Database)

Data Processing - BeautifulSoup – Web scraping, Requests – HTTP handling, DuckDuckGo Search API

DevOps & Deployment - Docker – Containerization, Docker Compose – Service orchestration, GitHub Actions – CI/CD pipeline, AWS EC2 – Cloud deployment

2.3 System Workflow

The system follows a **multi-stage intelligent workflow** that automates the research process from query to report generation.

Step 1: User Input

- User submits a research query via frontend

Step 2: API Request Handling

- Request is sent to API server (/api/research)
- API server forwards request to Python backend

Step 3: Research Agent Execution

- Performs web search using DuckDuckGo
- Retrieves relevant URLs

Step 4: Content Scraping

- Scraper extracts data from web pages
- Cleans and structures content

Step 5: Analysis Phase

- Analysis agent filters noise
- Extracts key insights and topics

Step 6: Report Generation

- Summary agent uses LLM (Flan-T5)
- Generates structured research report

Step 7: Storage & Indexing

- Report stored in JSON format

- Embeddings generated and stored in FAISS

Step 8: Response Delivery

- Backend returns job ID
- Frontend fetches results using polling
- Final report displayed to user

Fig. 2.3: System Workflow Diagram

Chapter 3: Implementation

3.1 Frontend Implementation

The frontend of the Energy Research App is developed using React with Vite, providing a fast and responsive user interface. The UI is styled using Tailwind CSS, ensuring a clean and modern design.

3.1.1 API Integration

The frontend communicates with backend services using a generated API client and a custom fetch wrapper.

- Base URL is configured using: setBaseUrl("http://<EC2-IP>:8080");
- All API calls are made using:
    - React Query (for caching & async handling)
    - Custom fetch wrapper for request control

3.1.2 Key Features Implemented

- Query input interface
- Research history display
- Report visualization
- Semantic search interface

3.2 Backend Implementation

The backend consists of two layers:

1. API Server (Node.js)
2. AI Backend (FastAPI)

3.2.1 API Server (Node.js - Express)

The API server acts as a gateway between frontend and AI backend.

Responsibilities:

- Route /api/* requests
- Forward to Python backend
- Handle CORS issues
- Maintain separation of concerns

Example: /api/research → forwarded to → Python /research

3.2.2 Python Backend (FastAPI)

The Python backend handles all AI processing and business logic.

Key Modules:

- main.py → Entry point
- agents/ → AI agents
- tools/ → Search & scraping utilities
- database/ → Storage & FAISS
- models/ → LLM integration

3.2.3 API Endpoints Implemented

Method	Endpoint	Description
GET	/healthz	Health check
POST	/research	Start research
GET	/research/{id}	Get result
GET	/history	Fetch reports
POST	/search	Semantic search

3.3 AI Agent Implementation

The core intelligence of the system is based on a multi-agent architecture, where each agent performs a specific task in the research pipeline.

3.3.1 Research Agent

- Performs web search using DuckDuckGo
- Collects top URLs
- Initiates scraping process

3.3.2 Scraper Tool

- Uses BeautifulSoup
- Extracts:
    - Paragraphs
    - Headings
    - Relevant content
- Handles multiple URLs concurrently

3.3.3 Analysis Agent

- Filters irrelevant data
- Extracts key insights

- Identifies main topics

3.3.4 Summary Agent

- Uses HuggingFace Flan-T5 model
- Generates structured reports:
    - Introduction
    - Key insights
    - Conclusion

3.3.5 Embedding & Storage

- Sentence Transformers generate embeddings
- Stored in FAISS for semantic search
- Reports saved in JSON format

3.3.6 Agent Pipeline Flow

The agents operate sequentially:

1. Research Agent → gathers data
2. Scraper → extracts content
3. Analysis Agent → processes data
4. Summary Agent → generates report

Chapter 4: DevOps & Deployment

4.1 Dockerization

To ensure portability, consistency, and scalability, the entire application is containerized using **Docker**. Each major service runs inside its own isolated container.

4.1.1 Multi-Container Architecture

The system is divided into three Docker services:

- **Frontend Container**
  - Built using React + Vite
  - Served via Nginx
  - Runs on port 80
- **API Server Container**
  - Node.js (Express)
  - Handles routing and proxy logic
  - Runs on port 8080
- **Python Backend Container**
  - FastAPI-based AI engine
  - Executes multi-agent pipeline
  - Runs on port 8000

4.1.2 Dockerfile Strategy

Each service has its own optimized Dockerfile:

Frontend

- Multi-stage build:
  - Build stage → Node.js (pnpm build)
  - Production stage → Nginx (serve static files)

Backend

- Lightweight Python image
- Installs dependencies via requirements.txt
- Runs FastAPI using Uvicorn

4.1.3 Docker Compose Orchestration

All services are managed using **Docker Compose**, enabling:

- Service networking

- Environment isolation
- Easy startup/shutdown

Example:

```
services:
  frontend:
  api-server:
  python-backend:
```

4.1.4 Benefits of Dockerization

- Environment consistency (works on any system)
- Easy deployment on cloud
- Isolation of services
- Simplified dependency management

4.2 CI/CD Pipeline (GitHub Actions)

The project implements a **Continuous Integration and Continuous Deployment (CI/CD)** pipeline using **GitHub Actions**.

4.2.1 Pipeline Overview

The pipeline automates:

- Code integration
- Docker image building
- Image publishing
- Remote deployment

4.2.2 Workflow Steps

1. Developer pushes code to GitHub (main branch)
2. GitHub Actions is triggered
3. Docker images are built:
    - energy-frontend
    - energy-api-server
    - energy-python-backend
4. Images are pushed to Docker Hub
5. EC2 instance is accessed via SSH
6. Latest images are pulled
7. Containers are restarted using Docker Compose

4.2.3 Deployment Script Logic

Key operations:

```
docker pull <images>
docker-compose down
docker-compose up -d
docker image prune -af
```

4.2.4 Key Features of Pipeline

- Fully automated deployment
- Zero manual server intervention
- Consistent production updates
- Integrated cleanup (prevents disk overflow)

4.3 AWS EC2 Deployment

The application is deployed on a **cloud-based virtual server using AWS EC2**, enabling remote access and real-time usage.

4.3.1 EC2 Instance Setup

- OS: Ubuntu Linux
- Docker & Docker Compose installed
- Security groups configured:
    - Port 80 → Frontend
    - Port 8080 → API Server
    - Port 8000 → Backend

4.3.2 Deployment Process

1. EC2 instance is launched
2. SSH access configured using key pair
3. Docker environment installed
4. Application deployed via CI/CD pipeline
5. Services started using Docker Compose

4.3.3 Application Access

- Frontend accessed via:

  http://<EC2-PUBLIC-IP>

- API server:

  http://<EC2-PUBLIC-IP>:8080

4.3.4 Advantages of Cloud Deployment

- Global accessibility
- Scalability
- High availability
- Real-time updates via CI/CD

4.4 Summary of DevOps Implementation

The integration of Docker, CI/CD, and AWS enables:

- Automated deployment pipeline
- Scalable architecture
- Reduced manual errors
- Faster development-to-production cycle

Chapter 5: Database & Storage

5.1 FAISS Vector Database

The system uses FAISS as a vector database to enable semantic search over stored research reports.Traditional databases rely on keyword matching, which is limited. FAISS allows:

- Semantic similarity search
- Faster retrieval of relevant information
- Efficient handling of high-dimensional embeddings

5.1.1 Working Principle

1. Text data is converted into vector embeddings using Sentence Transformers
2. These embeddings are stored in FAISS index
3. When a query is received:
    - It is converted into a vector
    - FAISS finds closest matching vectors using similarity search

5.1.2 Advantages

- High-speed similarity search
- Scalable for large datasets
- Enables intelligent search instead of keyword matching

5.2 JSON Storage System

In addition to FAISS, the system uses a JSON-based storage mechanism for storing complete research reports.

- Store structured research outputs
- Maintain history of generated reports
- Provide quick retrieval without recomputation

5.2.2 Storage Structure

Each report is stored as a JSON file containing: Report ID, Query, Generated summary, Sources, Timestamp

Example structure:

```
{
  "id": "report_id",
  "query": "What is renewable energy?",
  "summary": "...",
  "sources": ["url1", "url2"],
  "timestamp": "2026-04-21"
}
```

5.2.3 Advantages

- Lightweight and easy to manage
- Human-readable format
- No need for complex database setup
- Suitable for small-to-medium scale systems

5.3 Data Flow Management

The system ensures efficient data flow between different components, from user input to final output storage and retrieval.

5.3.1 End-to-End Data Flow

1. User submits query
2. Backend processes request via agents
3. Data is: Stored as JSON (report), Converted to embeddings (vector)
4. Stored in: JSON storage (for retrieval), FAISS index (for semantic search)

5.3.2 Data Lifecycle

- **Input Stage** → Query received
- **Processing Stage** → Agents analyze data
- **Storage Stage** → JSON + FAISS
- **Retrieval Stage** → API endpoints fetch data

5.3.3 API Integration

Data is accessed via endpoints:

- /history → Fetch all reports
- /history/{id} → Fetch specific report
- /search → Semantic search using FAISS

5.3.4 Synchronization Between Storage Systems

- Each JSON report has a corresponding embedding
- FAISS stores vector references mapped to report IDs
- Ensures consistency between:
    - Structured data (JSON)
    - Semantic data (FAISS)

Chapter 6: Results and Discussions

6.1 System Output

The Energy Research App successfully generates structured research reports from user queries using its multi-agent pipeline.

6.1.1 Output Generation

When a user submits a query:

- The system processes it through multiple agents
- Extracts relevant information from web sources
- Generates a structured report containing:
    - Introduction
    - Key insights
    - Summary
    - References

6.1.2 Sample Output Description

For example, for the query:

"What is energy?"

The system produces:

- Definition and explanation
- Types of energy
- Real-world applications
- References from multiple sources

6.1.3 Frontend Output Display

The frontend displays:

- Research progress (live updates)
- Final structured report
- History of previous searches
- Semantic search results

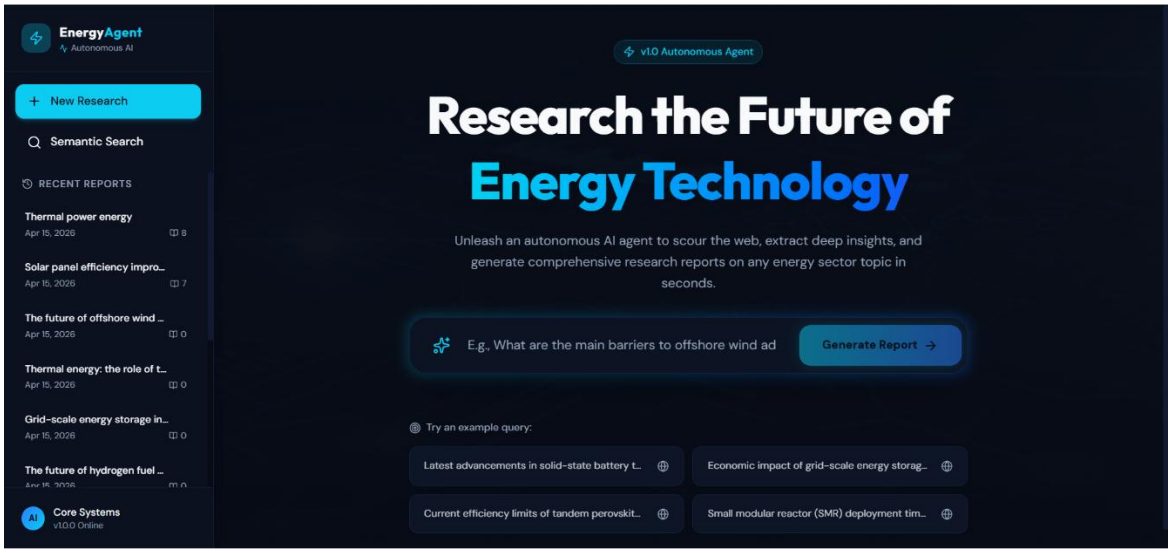

Fig. 6.1: User Interface – Query Input Page

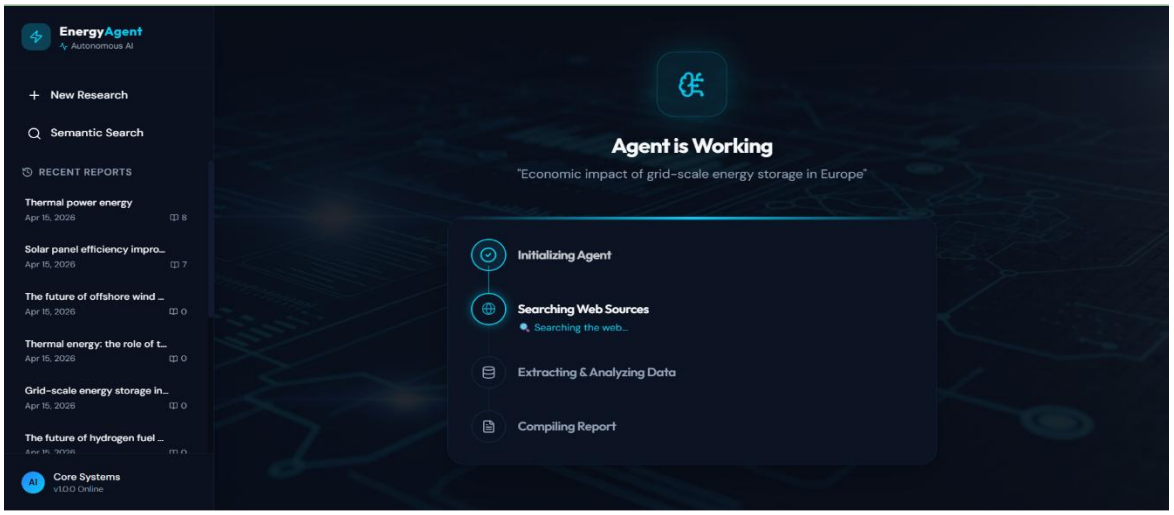

Fig 6.2: User Interface – Query Parsing Page

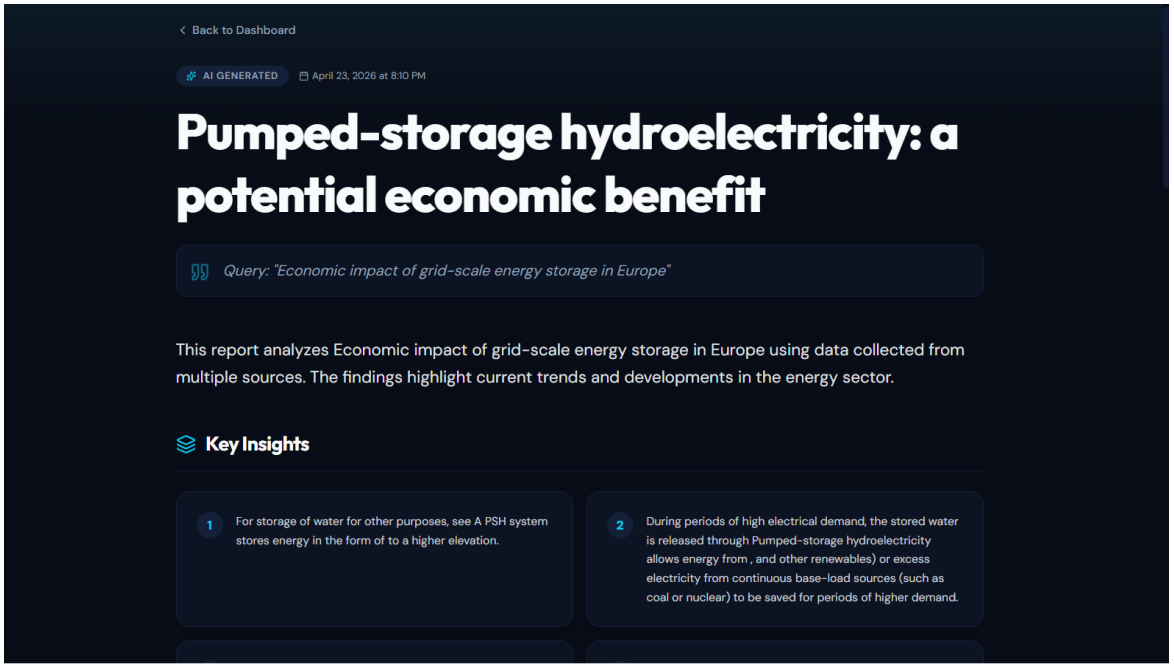

Fig 6.3: Generated Research Report Output Page (a)

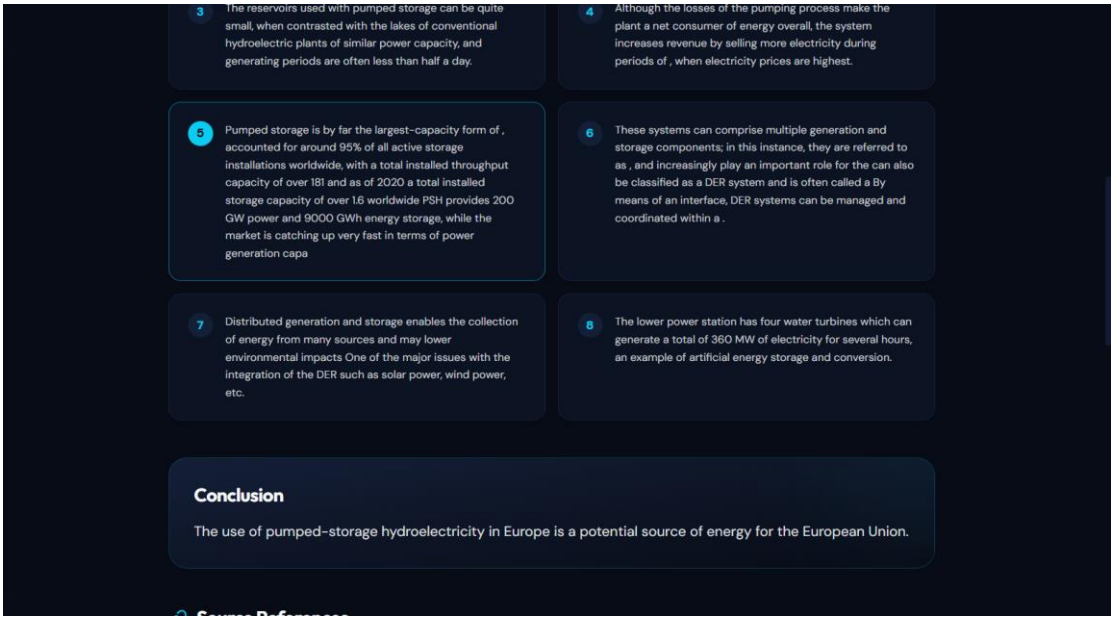

Fig 6.4: Generated Research Report Output Page (b)

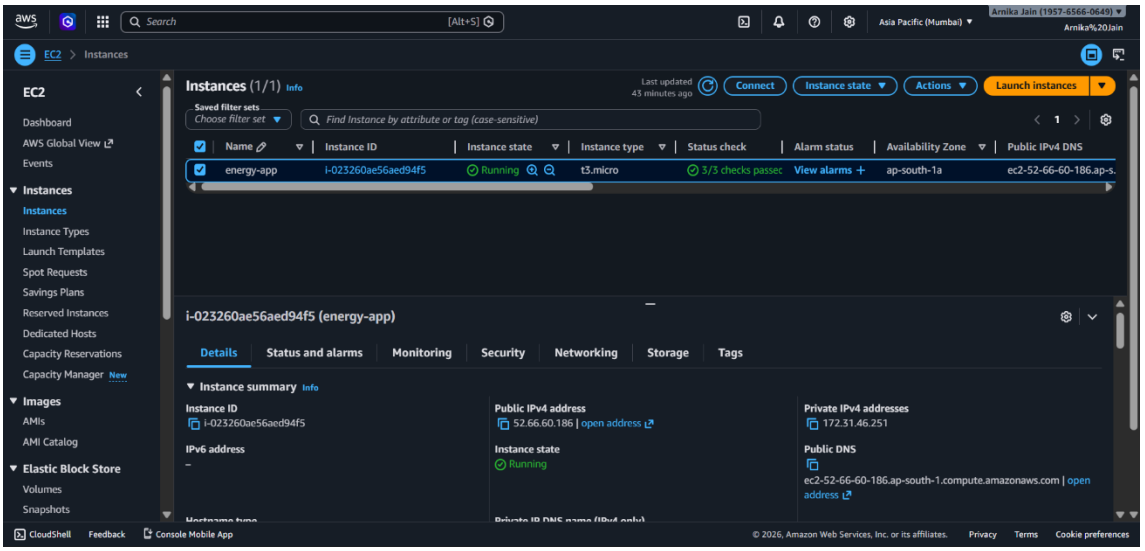

Fig 6.5: AWS EC2 Instance

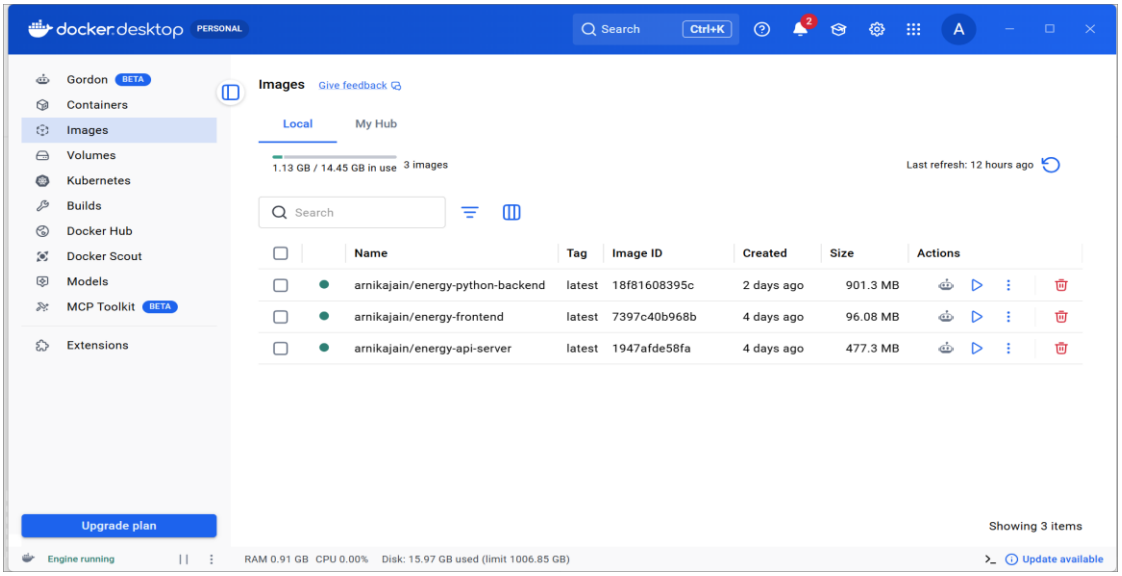

Fig 6.6: Docker Desktop

6.1.4 Observations

- System successfully integrates multiple components
- Reports are generated in structured format
- Real-time updates improve user experience

6.2 Performance Analysis

The performance of the system was evaluated based on response time, accuracy, and system efficiency.

6.2.1 Response Time

Observation:

- Most delay occurs in: Web scraping, LLM processing

6.2.2 System Efficiency

- Concurrent scraping improves speed
- Docker ensures consistent performance
- CI/CD reduces deployment time

6.2.3 Scalability

- Docker-based architecture allows scaling
- Can handle multiple users with container scaling
- FAISS supports large dataset search

6.3 Challenges Faced

During development and deployment, several technical challenges were encountered.

6.3.1 API Routing Issues

Problem: 404 errors when frontend tried to access backend

Cause: Incorrect base URL and missing /api routing

Solution: Fixed API proxy configuration, Corrected base URL in frontend

6.3.2 Deployment Issues on Cloud

Problem: Application worked locally but not on EC2

Cause: Environment mismatch and port configuration

Solution: Updated Docker configuration, Corrected exposed ports and URLs

6.3.3 Disk Space Issues

Problem: EC2 storage getting full due to multiple images

Solution: Implemented cleanup command: docker image prune -af

Chapter 7: Summary and Conclusion

7.1 Summary

This project presented the design and implementation of an AI-powered Energy Research Application that automates the process of information gathering, analysis, and report generation.

The application follows a multi-agent architecture, where:

- The Research Agent collects data
- The Analysis Agent processes content
- The Summary Agent generates structured reports

Additionally, the system incorporates:

- Web scraping techniques for real-time data extraction
- LLM-based summarization using transformer models
- Cloud deployment on AWS EC2 for accessibility

7.2 Conclusion

The Energy Research App successfully demonstrates how Artificial Intelligence, Agentic Systems, and DevOps practices can be integrated into a unified application.

Key conclusions drawn from the project:

- The system is capable of generating accurate and structured research outputs
- The multi-agent approach improves modularity and efficiency
- Semantic search using FAISS enhances data retrieval capabilities
- Dockerization and CI/CD enable smooth and automated deployment
- The system is scalable and production-ready

Despite minor challenges during development, such as API routing issues and deployment errors, the final system operates reliably and efficiently.

Overall, the project showcases a modern AI-driven application architecture combining Machine Learning, Web Technologies, Cloud Deployment, Automation (DevOps)

Chapter 8: Future Scope

The current system provides a strong foundation for automated research; however, several enhancements can be implemented to improve functionality and scalability.

1 Advanced AI Models

- Integration of more powerful LLMs (e.g., GPT-based or larger transformer models)
- Improved summarization quality and contextual understanding

2 Real-Time Data Integration

- Use of live APIs for real-time data sources
- Integration with academic databases (e.g., research papers)

3 User Authentication System

- Login and user-specific dashboards
- Personalized research history

4 Scalability Improvements

- Deployment using Kubernetes for container orchestration
- Load balancing for handling multiple users

5 Enhanced Semantic Search

- Hybrid search (keyword + vector search)
- Improved ranking algorithms

6 UI/UX Improvements

- Interactive dashboards
- Visualization of research data (charts, graphs)

7 Mobile Application

- Development of Android/iOS version
- Cross-platform support

8 Integration with External Tools

- Integration with citation tools
- Export reports as PDF or DOCX

9 Performance Optimization

- Caching mechanisms
- Faster scraping techniques
- Model optimization

10 Security Enhancements

- API authentication (JWT)
- Secure data handling

Appendix

Appendix A: Sample API Endpoints

Method	Endpoint	Description
GET	/healthz	Health check
POST	/research	Start research
GET	/research/{id}	Get result
GET	/history	Fetch reports
POST	/search	Semantic search

Appendix B: Sample Docker Commands

```
docker compose up -d
docker compose down
docker image prune -af
```

Appendix C: Project Structure

```
artifacts/
 ├── python-backend/
 ├── api-server/
 └── energy-researcher/
```

Bibliography

[1] T. Brown et al., “Language Models are Few-Shot Learners,” Advances in Neural Information Processing Systems, 2020.

[2] Docker Inc., “Docker Documentation,” Available: https://docs.docker.com/

[3] Amazon Web Services, “Amazon EC2 Documentation,”

Available: https://docs.aws.amazon.com/ec2/

[4] DuckDuckGo, “DuckDuckGo Search API,” Available: https://duckduckgo.com